



PDF Download
3694715.3695955.pdf
29 March 2026
Total Citations: 4
Total Downloads: 1514

Latest updates: <https://dl.acm.org/doi/10.1145/3694715.3695955>

RESEARCH-ARTICLE

Scaling Deep Learning Computation over the Inter-Core Connected Intelligence Processor with T10

YIQI LIU, University of Illinois Urbana-Champaign, Urbana, IL, United States

YUQI XUE, University of Illinois Urbana-Champaign, Urbana, IL, United States

YU CHENG, Microsoft Research, Redmond, WA, United States

LINGXIAO MA, Microsoft Research, Redmond, WA, United States

ZIMING MIAO, Microsoft Research, Redmond, WA, United States

JILONG XUE, Microsoft Research, Redmond, WA, United States

[View all](#)

Open Access Support provided by:

[University of Illinois Urbana-Champaign](#)

[Microsoft Research](#)

Published: 15 November 2024

[Citation in BibTeX format](#)

SOSP '24: ACM SIGOPS 30th Symposium
on Operating Systems Principles
November 4 - 6, 2024
TX, Austin, USA

Conference Sponsors:
SIGOPS

Scaling Deep Learning Computation over the Inter-core Connected Intelligence Processor with T10

Yiqi Liu
yiqiliu2@illinois.edu
UIUC

Yuqi Xue
yuqixue2@illinois.edu
UIUC

Yu Cheng
yu.cheng@pku.edu.cn
Microsoft Research

Lingxiao Ma
lingxiao.ma@microsoft.com
Microsoft Research

Ziming Miao
ziming.miao@microsoft.com
Microsoft Research

Jilong Xue
jxue@microsoft.com
Microsoft Research

Jian Huang
jianh@illinois.edu
UIUC

Abstract

As AI chips incorporate numerous parallelized cores to scale deep learning (DL) computing, inter-core communication is enabled recently by employing high-bandwidth and low-latency interconnect links on the chip (e.g., Graphcore IPU). It allows each core to directly access the fast scratchpad memory in other cores, which enables new parallel computing paradigms. However, without proper support for the scalable inter-core connections in current DL compilers, it is hard for developers to exploit the benefits of this new architecture.

We present T10, the first DL compiler to exploit the inter-core communication bandwidth and distributed on-chip memory on AI chips. To formulate the computation and communication patterns of tensor operators in this new architecture, T10 introduces a distributed tensor abstraction *rTensor*. T10 maps a DNN model to execution plans with a generalized compute-shift pattern, by partitioning DNN computation into sub-operators and mapping them to cores, so that the cores can exchange data following predictable patterns. T10 makes globally optimized trade-offs between on-chip memory consumption and inter-core communication overhead, selects the best execution plan from a vast optimization space, and alleviates unnecessary inter-core communications. Our evaluation with a real inter-core connected AI chip, the Graphcore IPU, shows up to 3.3× performance improvement, and scalability support for larger models, compared to state-of-the-art DL compilers and vendor libraries.

CCS Concepts: • Software and its engineering → Compilers; • Computer systems organization → Parallel architectures; Neural networks.

Keywords: Deep Learning Compiler, Intelligence Processing Unit, Distributed Shared Memory, ML Accelerator

ACM Reference Format:

Yiqi Liu, Yuqi Xue, Yu Cheng, Lingxiao Ma, Ziming Miao, Jilong Xue, and Jian Huang. 2024. Scaling Deep Learning Computation over the Inter-core Connected Intelligence Processor with T10. In *ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP '24)*, November 4–6, 2024, Austin, TX, USA. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3694715.3695955>

1 Introduction

To meet the ever-increasing compute demand of deep learning (DL) workloads, various AI chips or intelligence processors have been developed [22, 32, 33]. Typically, an AI chip employs numerous cores to provide high compute throughput. Each core has a small SRAM as its local scratchpad memory. To exploit the parallelism across cores, DL compilers partition the computation into multiple pieces. To synchronize data across cores, all cores share a global memory backed by a high-bandwidth off-chip memory (e.g., HBM).

Unfortunately, the global memory bandwidth growth gradually lags behind the fast growth of computing performance. Instead of fetching all data from the global memory, inter-core communication links allow cores to directly reuse the data from each other, enabling higher on-chip data reuse. For example, unlike the TPU [21] and GPU [33] architectures shown in Figure 1, the Graphcore IPU [24] allows each core to access another core's local memory at 5.5GB/s. The 1,472 cores per chip yield an all-to-all transfer bandwidth of 8TB/s, much higher than the HBM bandwidth (1.94TB/s on an A100 GPU). The aggregated bandwidth can further scale when future technology fits more cores in a chip, making it a promising approach for breaking the memory bandwidth wall. Thus, the inter-core links are employed in many emerging accelerators, including Graphcore IPU [24], SambaNova SN10 [39], Cerebras WSE [27], and Tenstorrent Grayskull [49].

However, this new architecture makes executing DL models more complex. In the traditional *global shared-memory architecture*, programs access all data from a unified memory, so compilers only need to focus on partitioning computation among cores. In contrast, the on-chip inter-core links enable a *distributed on-chip memory architecture* from programmers' perspective, which requires compilers to coordinate the computation partitioning, data placement, and inter-core



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike International 4.0 License.

SOSP '24, November 4–6, 2024, Austin, TX, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1251-7/24/11

<https://doi.org/10.1145/3694715.3695955>

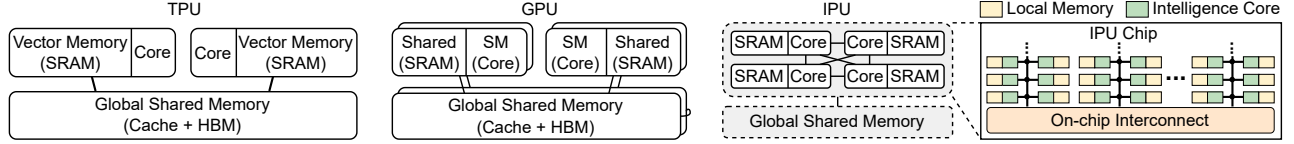


Figure 1. System architecture of TPU (left), GPU (middle), and IPU (right) chips.

communication. Simply employing existing compiler techniques causes unnecessary inter-core communication and data duplication in the precious on-chip memory (see §2.2 and Figure 2). As a result, emerging inter-core connected AI chips fail to compete with the traditional global shared-memory-based AI accelerators due to the inefficient compiler support, although the industry has invested hundreds of millions of dollars in hardware development [4, 12, 44].

In this paper, we present T10, a DL compiler for efficiently utilizing inter-core connected AI chips like Graphcore IPU. T10 scales DL computation with distributed on-chip memory by generating execution plans with a “compute-shift” paradigm. T10 carefully plans the tensor placement and transmission among cores to best utilize both inter-core connection bandwidth and on-chip memory capacity.

First, T10 introduces a distributed tensor abstraction called *RotatingTensor*, or *rTensor*, that rotates its partitioned tiles across the cores. T10 aligns the rotating paces of different tensors in an operator based on its computation logic and partitioning plan, so that data tiles and computation meet at the right timing in every step. *rTensor* leverages the compute-shift paradigm to formulate the inter-core communication patterns of DNN workloads, which differs from the traditional “load-compute-store” paradigm [45, 63] (i.e., load data from global shared memory, compute, and store the results back) used in global shared-memory architectures.

Second, as different tensor partitioning and rotating plans create different trade-offs among the memory footprint, computation time, and communication cost, T10 constructs an accurate cost model to guide the optimization process. The compute-shift computing paradigm facilitates the development of an accurate cost model, as it inherently avoids non-deterministic data accesses and allows software to explicitly manage the inter-core data transfers. T10 further eliminates the unpredictability in its cost model by exploiting the deterministic computation patterns of DNN workloads.

Third, to flexibly trade-off between memory footprint and communication overhead, T10 builds a spatial-temporal optimization space, where it divides an operator into multiple multi-step sub-operators and maps them to individual cores. T10 employs a cost-aware operator scheduling process to generate a range of execution plans with varying memory requirements. In consideration of the holistic model scheduling, it allows each operator to switch between memory- or compute-efficient plans during execution. T10 then applies a holistic reconciliation process to search for an optimized

end-to-end execution plan that can fit multiple operators or even the entire model into the distributed on-chip memory.

We implement T10 in 10K lines of code with Python, C/C++, and assembly, for the execution plan optimization and kernel code generation. We evaluate T10 with various DNN models, including CNNs and transformers with varying batch sizes. The models are executed on a real Graphcore IPU MK2 chip. T10 outperforms current DL compilers and vendor libraries by up to 3.3×, and allows much larger models and batch sizes to fit into the distributed on-chip memory. We also show its benefits in serving large language models. Overall, we make the following contributions in this paper:

- We develop T10, the first DL compiler to exploit the inter-core communication bandwidth and best utilize distributed on-chip memory on intelligence processors (§3).
- We propose a new tensor abstraction to formulate the inter-core transfer patterns of DNN models and represent them with the compute-shift execution plan (§4.1 & §4.2).
- We build an accurate cost model to guide the optimization for computation partitioning and inter-core communications, significantly reducing the compilation time (§4.3).
- We propose a cost-aware operator scheduling process to tradeoff memory footprint and communication overhead, and generate optimized end-to-end execution plan (§4.3).
- We implement T10 as a standalone compiler and develop generic device interface for enabling the mapping of tensor operators to inter-core connected AI chips (§4.4 & §5).
- We evaluate T10 with a real Graphcore IPU MK2 chip and show its efficiency and scalability for DNN workloads (§6).

2 Background and Motivation

We introduce the system architecture of inter-core connected intelligence processor and discuss the motivation of T10.

2.1 Inter-core Connected Intelligence Processor

As discussed in §1, many AI chips are adopting the inter-core connected architecture [24, 27, 39, 49]. In this paper, we focus on a representative example, the Graphcore Intelligence Processing Unit (IPU) MK2 [24], as shown in Figure 1 (right). An IPU chip has 1,472 cores, and each core executes independent threads in parallel with a private 624KB scratchpad memory, which adds up to a total of 896MB on-chip memory. Compared to the global shared memory architecture, a key distinction is that IPU cores are interconnected by high-bandwidth low-latency links. Each core can access the scratchpad memory of another core at 5.5GB/s, offering an aggregated inter-core all-to-all bandwidth of $1472 \times 5.5\text{GB/s} \approx 8\text{TB/s}$ [19].

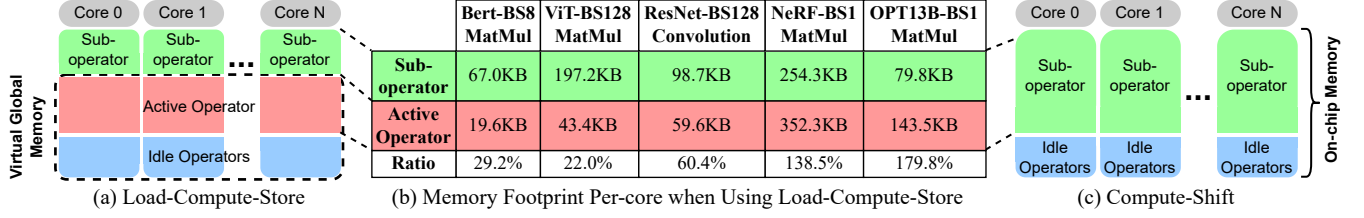


Figure 2. A comparison of the conventional load-compute-store (a) vs. our compute-shift (c) style execution. (b) shows the per-core memory footprint of representative operators when running DNN models on IPU using VGM. **Ratio** is the potential increase in sub-operator size by removing VGM. The result of OPT13B [57] comes from profiling one of its layers on IPU.

Specifically, cores can individually access data from other cores, without contending for the global shared memory. Thus, the inter-core connection effectively aggregates the local memories of all cores into a large distributed on-chip memory with 8TB/s total bandwidth, which is much higher than a global shared memory backed by off-chip HBM (e.g., 1.94TB/s on an A100 GPU). This can alleviate the well-known memory bandwidth bottleneck in DL applications.

2.2 Inefficiency of Existing Approaches

To support inter-core connected AI chips, existing compilers [59, 63] and libraries [14] mimic a shared memory for all cores by reserving a portion of local memory in each core and abstracting them as a “virtual global memory” (VGM), as shown in Figure 2 (a). By default, to store an entire DL model on chip, all tensors used by the operators of the model, including persistent weights and temporary activations, are placed in the VGM. During execution, the active operator (i.e., currently running operator) is partitioned into sub-operators, each running on one core. Tensors of the idle operators (i.e., operators stored on-chip but not currently running) are unused in the VGM. To execute a sub-operator, each core retrieves data from VGM to its local “sub-operator” memory (shaded in green in Figure 2), performs computation locally, and stores the result back to VGM. We define this as a “load-compute-store” paradigm.

Inefficient inter-core communications. VGM introduces significant inefficiencies in inter-core communications.

First, accessing tensor data from VGM causes *imbalanced memory accesses* across cores, where some cores issue or serve more data accesses than others. As each tensor partition is stored in one core but often used by multiple cores for computation, some cores can obtain needed partitions from their local memory, while other cores need to remotely fetch required partitions from their peers. Then, the execution is bottlenecked by cores that access remotely. Also, sub-optimal tensor placements may cause bandwidth contentions. For instance, when multiple cores access different data from the same core, these cores will contend for the limited 5.5GB/s bandwidth of a single core, stalling the entire execution.

Second, to store a tensor using the VGM, as shown by the red “Active Operator” boxes in Figure 2 (a), we split it into small pieces across multiple cores. To retrieve a complete

tensor, a core must fetch each piece from a different core, requiring it to communicate with multiple cores. This leads to *redundant inter-core communications*.

To quantify the inter-core communication overhead, we break down the data transfer and compute time for a modern compiler that uses VGM, such as Roller [63]. With VGM, the inter-core data accesses account for 50%–74% of the end-to-end execution time (see Figure 13). We will show how T10 reduces the overhead to 8%–43% by eliminating VGM and orchestrating the inter-core communications in §3 and §4.

Inefficient use of on-chip memory. The VGM uses the on-chip memory capacity inefficiently. As shown in Figure 2 (a), each sub-operator of the currently active operator loads required data from the VGM to its local memory, which duplicates the data in both memory spaces. To host the duplicated data, VGM reserves memory space on each core, as shown by the active operator region in Figure 2 (a). This leaves less free on-chip memory, restricts each core to accommodate a smaller sub-operator, and results in *low compute intensity*. With less data reuse inside a core, higher data transfer volume is required for performing the same computation.

We quantify the storage overhead of VGM for representative operators in Figure 2 (b). By removing the VGM (i.e., merging the active operator region into the sub-operator region) in Figure 2 (c), we can increase sub-operator size by 22%–180%. T10 leverages this to improve memory efficiency.

3 Core Idea of T10

To eliminate the excessive memory footprint and redundant inter-core communications of VGM, we map the DNN computation to a *compute-shift* pattern. In each step, each core independently computes a sub-task with data received from its upstream neighbors and shifts the data to its downstream. The feasibility of this approach for general DNNs comes from this observation: most DNN operators can be divided into regular computation tasks, which load and produce consecutive data tiles of the input and output tensors, respectively.

We show an example that maps a matrix multiplication (MatMul) operator to two cores in Figure 3 (a). We first partition the operator along dimension m onto two cores in Figure 3 (b). By default, both cores hold a copy of the weight tensor, which incurs memory capacity overhead. To reduce memory footprint, in Figure 3 (c), we further split the weight

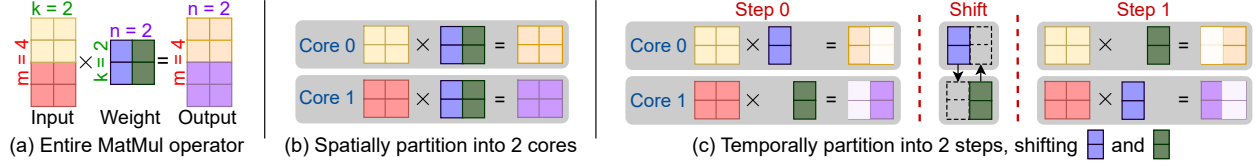


Figure 3. An example that maps a MatMul operator to two cores with the compute-shift style execution. Both (b) and (c) are valid compute-shift execution plans, but with different tradeoffs between memory footprint and communication overhead.

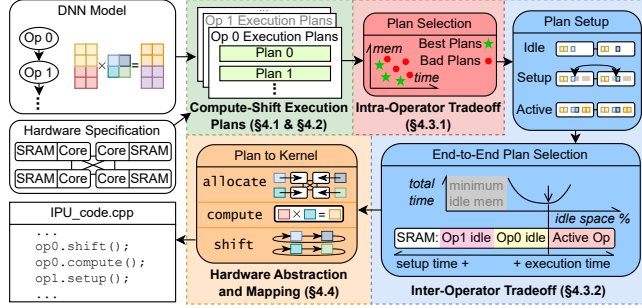


Figure 4. System overview of T10.

tensor along dimension n into two parts and place each part on one of the cores. Then, the computation must be conducted in two steps, as each core holds half of the weight tensor and performs half of its computation per step. Between the computation steps, each core circularly shifts its partition to the next core, forming a shift ring of two cores.

The compute-shift pattern avoids the inefficiencies of VGM. First, each part of the weight tensor is stored in at least one core at any time, which eliminates the need for a global memory to store shared data. This improves the memory capacity utilization and allows larger sub-operator sizes. Second, by circularly shifting tensors across cores, the communication volume is evenly distributed across the inter-core connections. Third, as we accurately align the computation with data tile (e.g., Figure 3 (c) shifts a 2×1 weight tile and then computes a 2×1 output tile), each core only needs to communicate with one other core for each tensor at each step, avoiding redundant communications to many cores.

To find the best execution plan with the compute-shift pattern, we must exploit *the tradeoff between memory footprint and communication overhead*. For example, both Figure 3 (b) and (c) show valid execution plans. Plan (b) finishes the entire computation in one step without inter-core communication, but has a higher memory footprint. Plan (c) has less memory footprint but incurs more communication overhead.

In reality, as we consider multi-dimensional DNN operators and thousands of cores on an IPU chip, deriving the best tradeoff can be difficult. An efficient compute-shift execution plan for them may contain numerous nested shift rings along multiple tensor dimensions, composing a massive tradeoff space to search through. Given limited inter-core connection bandwidth and on-chip memory capacity, we must also holistically tradeoff among multiple operators on the chip to derive an optimized end-to-end execution plan.

```
class RotatingTensor {
    vector< size_t > shape ;
    DataType         type ;
    vector< size_t > spatial_partition_factor ; // f_s
    vector< size_t > temporal_partition_factor ; // f_t
    vector< size_t > rotating_pace ;           // rp
};
```

Figure 5. r Tensor abstraction in T10.

Table 1. Terminology used in T10.

Symbol	Name	Description
f_s^X	Spatial Partition Factor	Spatially partitions a tensor X into sub-tensors.
f_t^X	Temporal Partition Factor	Temporally partitions a sub-tensor of X into sub-tensor partitions.
rp	Rotating Pace	Specifies how sub-tensor partitions are shifted among cores.
F_{op}	Operator Partition Factor	Spatially partitions an entire operator into sub-operators.

4 System Design of T10

We now introduce T10, a compiler designed to optimize end-to-end DNN model execution on an inter-core connected intelligence processor. We present the overview of T10 in Figure 4: (1) T10 introduces the RotatingTensor (r Tensor) abstraction to represent the partitioning and communication patterns of tensor operators on distributed on-chip memory (§4.1). (2) It uses r Tensor to map a DNN model to compute-shift execution plans. The rich expressiveness of r Tensor enables the tradeoff between memory usage and communication overhead (§4.2). (3) By configuring r Tensors in different ways, T10 defines a comprehensive optimization space for a DNN model. It adopts a two-staged optimization strategy to handle the tradeoff between inter-core communication and memory footprint, and optimize for end-to-end DNN model execution. For each operator, T10 finds the Pareto-optimal execution plans that represent the best trade-off between the execution time and memory footprint (§4.3.1). Then, T10 employs a holistic inter-operator memory reconciliation policy to determine the best end-to-end plan for the DNN model (§4.3.2). (4) The plan is compiled onto the processor using three abstracted device interfaces (§4.4).

4.1 r Tensor: A New Tensor Abstraction

To map a tensor onto the distributed on-chip memory and stream the partitioned sub-tensors across multiple groups of cores, T10 introduces a distributed tensor abstraction called RotatingTensor (r Tensor), as shown in Figure 5. In addition to defining the tensor shape and data type, r Tensor also describes how each tensor is partitioned, mapped, and shifted on the interconnected cores (summarized in Table 1).

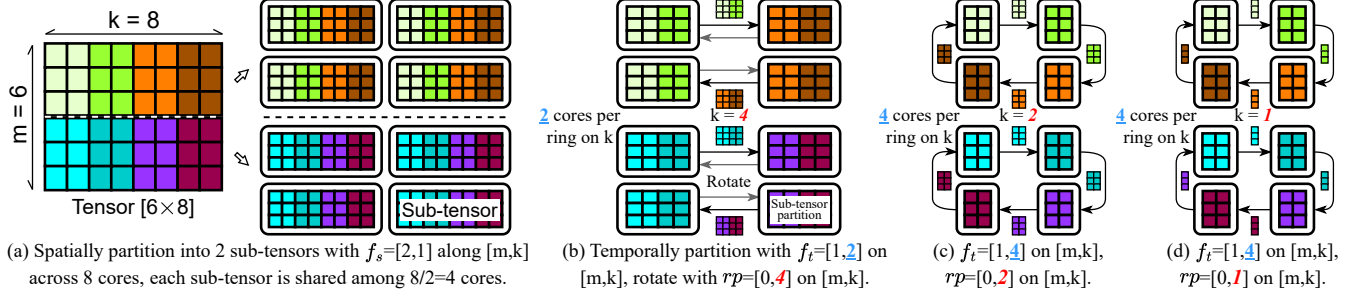


Figure 6. Illustration of r Tensor partitioning and rotating.

First, T10 partitions the computation of an operator onto multiple cores. Based on the data dependency, the computation partitioning will imply how each of its input/output tensor is partitioned. This gives a *spatial partition factor* (f_s), which splits a tensor into *sub-tensors*. Second, each sub-tensor may be required by multiple cores. To share a sub-tensor among them, we specify how the sub-tensor is further partitioned among the cores using a *temporal partition factor* (f_t). Third, we specify how the partitions of a sub-tensor are circularly shifted among the cores using the *rotating pace* (rp). Altogether, a set of r Tensors of an operator defines a compute-shift execution plan. The numerous possible r Tensor configurations of an operator generate a combinatorial optimization space of execution plans.

Specifically, f_s , f_t , and rp are vectors with a length equal to the number of dimensions of a tensor, indicating how the tensor is partitioned along each dimension. For example, in Figure 6 (a), a tensor T of shape $[6, 8]$ is partitioned onto 8 cores by a spatial factor $f_s = [2, 1]$, forming 2 sub-tensors of shape $[3, 8]$. Thus, to share each sub-tensor among 4 cores without incurring high memory footprint, a temporal factor $f_t = [1, 2]$ further partitions each sub-tensor into 2 partitions with shape $[3, 4]$, as shown in Figure 6 (b). It forms $\frac{4}{2} = 2$ rotation rings with 2 cores in each, where cores share the sub-tensor by circularly shifting its partitions. In comparison, Figure 6 (c) shows how another $f_t = [1, 4]$ splits the same sub-tensor to 4 partitions, on $\frac{4}{4}=1$ rotation ring with 4 cores.

Finally, the rotating pace rp controls how fast an r Tensor rotates, so we can align the data shifting with computation (see §4.2). Practically, rp specifies the number of data elements shifted in each step along each tensor dimension. For example, $rp = [0, 2]$ in Figure 6 (c) means that for each step, the sub-tensor shifts 2 elements along the second dimension (i.e., a data tile of shape $[3, 2]$), and finishes a full cycle in $\frac{8}{2}=4$ steps. Notably, different rps can be applied to the same set of f_s and f_t . For instance, in Figure 6 (d), $rp = [0, 1]$ shifts a tile of $[3, 1]$ for each step, requiring $\frac{8}{1}=8$ steps in total.

4.2 Compute-Shift Execution Plan

Using the r Tensor abstraction, T10 organizes the computation of a general DNN operator into a compute-shift pattern, where the operator’s computation and tensors are partitioned to individual cores and their local memories. The

entire computation involves multiple compute-shift steps until each tensor has been shifted across all cores. Each compute step is defined as a **sub-task**. In each compute-shift step, each core computes a sub-task and shifts local tensors to its neighbors. We now discuss how T10 partitions DNN operators into compute-shift-based execution plans.

Operator representation. To represent an operator’s computation, T10 uses tensor expression [5, 42, 51, 59, 63], which defines how each output tensor value is computed from the input values. For example, a matrix multiplication of tensors A in shape $[M, K]$ and B in $[K, N]$ into C is defined as

$$C[m, n] += A[m, k] * B[k, n], \quad (1)$$

where m , k , and n are axes to index the elements in each tensor. Equation (1) indicates that any value in C indexed by m and n (i.e., $C[m, n]$) is computed by summing $A[m, k] * B[k, n]$ over all possible indices k . T10 supports all common operators, like MatMul and Convolution, from DNN workloads in both inference and training. For a few special cases like Sort, which cannot be represented in tensor expression, T10 uses the implementations from the vendor library.

Partitioning an operator. To map an operator to interconnected cores, T10 first partitions it into parallel **sub-operators** along all unique axes in its tensor expression, using an *operator partition factor* (F_{op}). For example, Equation (1) contains axes m , k , and n , then F_{op} is a vector of three integer factors specifying how the three axes are spatially partitioned. The total number of sub-operators is the product of all elements in F_{op} . For example, $F_{op} = [2, 1, 3]$ for $[m, k, n]$ slices the operator into 6 sub-operators on 6 cores, each computing a $[\frac{M}{2}, \frac{K}{1}] \times [\frac{K}{1}, \frac{N}{3}]$ sub-matrix multiplication.

Partitioning r Tensors. T10 then uses F_{op} to derive the spatial partition factor f_s for each tensor, following the data dependencies in tensor expression. With the same example, for $F_{op} = [2, 1, 3]$ on $[m, k, n]$, the spatial partition factor for the tensor A is $f_s^A = [2, 1]$ for axes m and k . Similarly, for tensors B and C , we have $f_s^B = [1, 3]$ and $f_s^C = [2, 3]$.

If a tensor’s dimensions do not include some axis in F_{op} , each of the sliced sub-tensors is required by multiple sub-operators along the missing axis. Thus, once the spatial factor determines the number of cores that will share a sub-tensor, the temporal factor determines how we split the sub-tensor across these cores into rotation ring(s). In the above example,

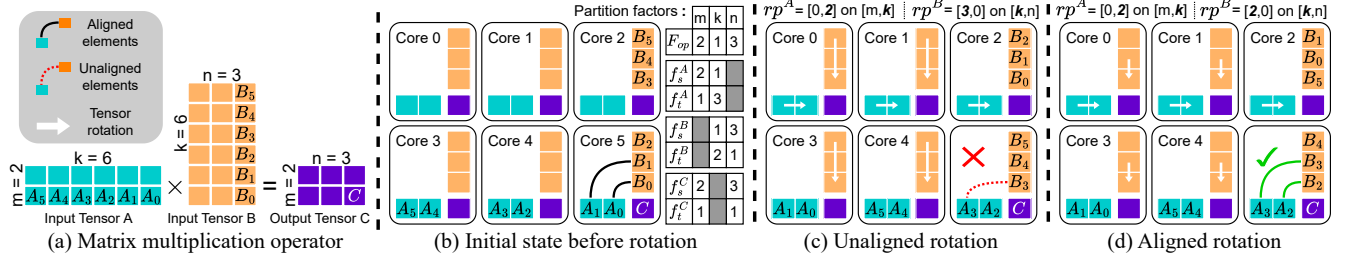


Figure 7. An example of the rotation of r Tensor. The compute-shift executions of the sub-operators need to be aligned.

F_{op} partitions the entire operator onto $2 \times 1 \times 3 = 6$ cores, and f_s^B spatially partitions tensor B into $1 \times 3 = 3$ sub-tensors. Thus, each sub-tensor is shared by $P = \frac{6}{3} = 2$ cores. Then, a temporal factor $f_t^B = [2, 1]$ further splits each sub-tensor into $2 \times 1 = 2$ partitions, forming $\frac{P}{2} = 1$ rotation ring.

T10 enforces that the product of elements in f_t , or $\prod f_t$, is a divisor of the number of cores that shares the sub-tensor (P), so that the number of rotation rings (i.e., $\frac{P}{\prod f_t}$) is an integer. If there is more than one rotation ring, we replicate each sub-tensor $\frac{P}{\prod f_t}$ times to ensure that each ring shares one copy of the sub-tensor. While the duplication consumes memory space, it may reduce the number of rotation steps by allowing a larger sub-task on each core at each step, which enables a trade-off between memory usage and communication cost. **Aligning the rotations of r Tensors.** Since a general DNN operator can have various tensor shapes, a naive partitioning plan can easily cause the tensor shifting and the sub-task computing at an unaligned pace. In Figure 7 (a), we still use the MatMul operator in Equation (1) as an example. We partition it into a 2×3 grid in Figure 7 (b), with the specified partition factors. Note that both A and B are temporally partitioned along axis k , but with different f_t factors.

The rotating paces of tensors in one operator must be aligned to ensure correct data dependency. In Figure 7 (c), tensors A and B are shifted with different rp s along axis k , which breaks data dependency. In the bottom-right core, we cannot compute $C += A_2 * B_2$, as B_2 is not on this core after an unaligned rotation. Thus, T10 synchronizes the rp of each r Tensor in Figure 7 (d). With $rp = 2$ on axis k for tensors A and B, in each step, each core shifts A and B for 2 data elements along k , and computes a sub-task whose length along k is also 2 (i.e., a sub-MatMul of shape $[m=1, k=2, n=1]$), requiring $\frac{6}{2} = 3$ steps to finish the sub-operator on this core.

To organize the computation into an aligned compute-shift plan, T10 enforces two constraints. First, if a set of r Tensors rotate along the same axis k , they must share the same rotating pace rp along k . Second, for each r Tensor, the rp value cannot exceed the length of its sub-tensor partition on dimension k , so that each rp -aligned sub-task can be executed on the sub-tensor partitions locally on each core. As shown in Figure 7, tensor A and B are partitioned by their f_t along k into dimension lengths of $\frac{6}{3} = 2$ and $\frac{6}{2} = 3$, respectively, so their rp on k should not be greater than 2.

To maximize compute intensity, T10 designates the rp as the minimum of the sub-tensor partition lengths.

With the above constraints, we can organize an operator's computation into a valid compute-shift execution plan. At each step, each sub-operator computes a sub-task partitioned by F_{op} and the rotating pace rp along each axis. Each sub-operator iterates over all its sub-tasks by nested-looping through the axes of this operator. Between sub-tasks, an r Tensor is rotated along the currently iterating axis for all its sub-tensors, until all sub-tasks are enumerated.

4.3 Intra-operator and Inter-operator Trade-off

For each operator, there could be a vast number of execution plans involving different spatial and temporal partition factors and rotating paces. Moreover, an end-to-end model consists of numerous operators, creating a substantial combinatorial optimization space. T10 defines a *two-level trade-off space between execution time and memory consumption*.

First, when determining each operator's execution plan, we can trade memory space for execution efficiency by specifying a smaller temporal partitioning factor. This can reduce communication costs by reducing the hops in the rotation loop, while at the cost of using more memory to hold duplicated tensors. We refer to this as **intra-operator trade-off**.

Second, we can tradeoff between memory space and execution time across all operators holistically when deciding the end-to-end model execution plan. Different operators have different memory-latency trade-offs, allowing us to allocate more memory to operators with higher memory-cost efficiency. Moreover, a single operator can have multiple execution plans, such as utilizing a memory-efficient plan when the operator is not executing to save more memory space, and switching to an execution-efficient plan when it is about to execute. We refer to this as **inter-operator trade-off**.

To optimize such two-level trade-offs, T10 decouples the combinatorial space into two distinct stages. First, T10 optimizes each individual operator's execution by searching for all Pareto-optimal trade-off plans between execution time and memory consumption (§4.3.1). Second, T10 globally optimizes the memory allocation among different operators based on the intra-operator Pareto-optimal plans (§4.3.2).

4.3.1 Searching Pareto-optimal Intra-operator Plans.

For each operator, to search for the optimal execution plan

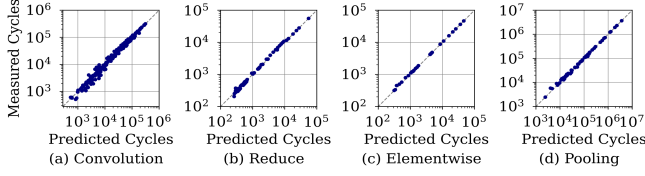


Figure 8. Cost model accuracy for different operator types and shapes. Each point represents the measured vs. predicted execution time of a sub-operator.

among numerous configuration choices, an efficient performance feedback for each plan is essential. T10 leverages the unique advantage of distributed on-chip memory architecture, which involves only local computation and communication, to design a sufficiently accurate cost model.

First, given the partitioning factors of an operator and its tensors, for each plan we can statically derive performance factors such as per-core computation task, per-step communication volume, and memory footprint. Second, the computation on each core in each step only consumes data from local memory, preventing unpredictable memory stalls from lower-level memory. To build a cost model for each operator type, we randomly generate sub-tasks with different shapes, run them on a single IPU core, and profile their execution time. We fit a linear regression model using the sub-task shape as input and the execution time as output. Also, an interface is exposed for users to implement custom cost functions for their custom kernels. In addition, the communication time is also accurately fitted by a linear regression model that takes the data transfer volume as input.

With the cost model, T10 quickly examines each execution plan and chooses the best candidates that sit on the Pareto optimal trade-off curve, where each plan either runs faster than any other plans with the same or less memory footprint or uses less memory than any others with the same or lower execution time (see Figure 17 in our evaluation for details).

Cost model accuracy. To test the accuracy of the cost models, we vary the operator shape and compare the predicted execution time with the actual profiled execution time. Figure 8 shows the accuracy of representative operator types. For most operators, T10 achieves near-perfect accuracy. The only exception is convolution, which is implemented with vendor-supplied kernels that apply some black-box optimizations. Even with slight inaccuracy, T10 can still find sufficiently good execution plans and outperform state-of-the-art compilers (see §6.3). We envision that hardware vendors would be able to supply a perfect cost model for their kernels as they integrate T10 in their toolchain.

Search constraints. When T10 enumerates all possible execution plans, a large portion of plans are evidently inefficient, and it is unnecessary to evaluate them with the cost model. Thus, T10 employs a rule-based approach to filter out inefficient plans, by applying two user-configurable constraints.

First, a plan utilizing too few cores will cause compute underutilization. Thus, the *parallelism constraint* specifies

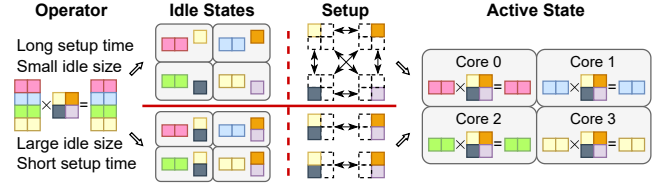


Figure 9. Switching operator state from idle to active. There is a tradeoff between the memory footprint of idle states vs. the time of turning idle states to active states (setup time).

the minimum number of cores an operator uses, filtering out plans with low parallelism. For example, to run a 1-dimension operator with dimension length L on C cores, we may partition it into 1 to $\min(L, C)$ sub-operators, which utilizes 1 to $\min(L, C)$ cores. Thus, there are $\min(L, C)$ possible values for F_{op} . To utilize at least 90% of the cores, we must partition it into $0.9 \times \min(L, C)$ to $\min(L, C)$ sub-operators, so we only need to enumerate $\frac{\min(L, C)}{10}$ possible values for F_{op} .

Second, to leverage the matrix accelerator unit (e.g., AMP in IPU [13]) in each core, we may need to pad the tensor shape to align with hardware, which underutilizes the memory capacity and FLOPS. Thus, the *padding constraint* specifies the maximum padding as the ratio between the original tensor size and the padded tensor size, filtering out plans with excessive padding. For example, when partitioning a dimension with length L into p partitions with length l , we calculate the ratio between the original length and padded length as $\frac{l}{lp}$. Plans with ratios below a certain threshold (e.g., a threshold of 0.9 means that the max padding overhead is $\frac{1}{0.9} - 1 = 11\%$) will be discarded.

Only the remaining plans are evaluated by the cost model. We examine the impact of both constraints in §6.3.

4.3.2 Holistic Inter-operator Memory Reconciliation.

To execute a DNN model, T10 adopts the common approach of fitting multiple operators into on-chip memory, while only transferring the input and output data through off-chip memory. This strategy is advantageous as adjacent operators can reuse full intermediate data in the fast on-chip memory.

Given the limited on-chip memory, each operator in T10 is assigned two execution plans — one for minimum memory usage before operator execution (called *idle state*) and one for minimum latency during execution (called *active state*). When an operator begins to execute, a *plan setup phase* transforms the partitioning plan from idle state to active state, i.e., by transferring the necessary data through the inter-core connection. This creates a tradeoff between memory usage at idle state and the overhead at the setup phase. Figure 9 shows the state transition of a MatMul operator.

As each operator has many choices for idle and active plans, T10 trades-off globally with a holistic inter-operator reconciliation policy, as shown in Algorithm 1. Initially, T10 assigns the memory-efficient plan as the idle plan for all operators (line 3). The remaining memory space after placing

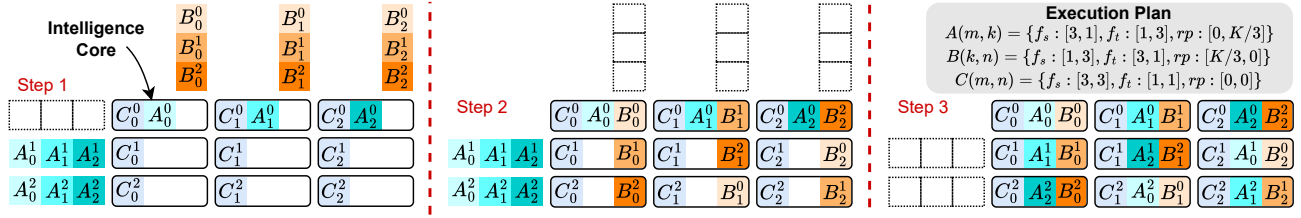


Figure 10. Sub-tensor placement for a matrix multiplication operator on 3×3 cores.

Algorithm 1: Inter-op memory reconciliation policy.

```

1 Function get_best_idle_configs(all_ops):
2   // start from the memory-efficient plan
3   for op in all_ops do
4     idle_plan[op]  $\leftarrow$  plan with min memory use
5     idle_mem_size  $\leftarrow \sum_{op \in all\_ops} idle\_plan[op].size$ 
6     cur_best_time  $\leftarrow \infty$ 
7   while idle_mem_size < max_mem_per_core do
8     // update active plan for each op
9     for op in all_ops do
10      active_plan[op]  $\leftarrow$  fastest plan that fits in mem
11      time  $\leftarrow estimate\_total\_time(idle\_plan, active\_plan)$ 
12      if time < cur_best_time then
13        cur_best_time  $\leftarrow time$ 
14        cur_best_plan  $\leftarrow (idle\_plan, active\_plan)$ 
15      // find the best operator and increase its idle mem size
16      best_op  $\leftarrow$  op with highest  $-\Delta T_S / \Delta M_I$ 
17      update idle_plan[best_op] with new plan
18      update idle_mem_size based on the new idle_plan
19   return cur_best_plan

```

all operators is considered as the *active memory*¹, which is used for executing the sub-operators. T10 then searches for the best execution plan by greedily trading more active memory space for *idle memory* space, aiming to reduce more setup time at the cost of slightly increasing execution time.

For each search step, T10 first selects the best operator, which has another execution plan that reduces the most setup time while adding the smallest idle space. This plan is found by computing the ratio $\Delta T_S / \Delta M_I$ for each operator, where ΔT_S and ΔM_I are the reduced setup time and the increased idle space (line 13). To apply the plan change, the active memory space is subtracted by ΔM_I (line 15). T10 updates the total execution time by finding the fastest execution plan that fits in the given active memory for each operator (line 8), and adding up the latencies of all operators (line 9).

The inter-operator scheduling policy of T10 can explore the complex search space with low algorithmic complexity. While there are $\prod_{i=0}^{\#ops} (op[i].num_idle_plans)$ idle plan combinations in total, we acquire an optimized one by searching only $\sum_{i=0}^{\#ops} (op[i].num_idle_plans)$ promising combinations.

¹Active memory is the memory space used by the operator currently in active state, and idle memory is the space used by operators in the idle state.

Given the moderate number of possible trade-off configurations, T10 currently searches all steps and chooses the one that can lead to the minimum end-to-end execution time.

4.4 Mapping to the Hardware Accelerator

The compilation approach of T10 is designed to be extensible for general distributed on-chip memory-based accelerators, which can be abstracted into a unified architecture with multiple cores, each equipped with dedicated local memory and interconnected via a high-speed on-chip network.

Abstracted device interface. T10 abstracts three key device interfaces: (1) allocate serves as a compile-time interface to allocate memory space for placing tensor partitions. (2) compute functions as a code generation interface that emits instructions for computing a specific sub-operator on a core. By default, T10 utilizes a few pre-defined code templates to generate single-core computing logic for each specific partition configuration. (3) shift serves as a runtime communication primitive to transmit a sub-tensor to the specified destination core. T10 leverages these interfaces to map the optimized execution plan to an accelerator.

Sub-tensor placement. T10 allocates the entire memory space and assigns each sub-tensor to its corresponding core. To optimize memory consumption, T10 performs tensor liveness analysis to reuse the memory of precedent operators. To ensure that sub-tensors from different tensors are in the same core at each rotating step, T10 arranges the initial placement of each tensor partition step-by-step by analyzing the computing order of each sub-operator and their data dependencies. This ensures that (1) the initial placement of all sub-tensor partitions satisfies the data dependency on each core, and (2) sub-tensor partitions along each axis are in ascending order, guaranteeing that the data dependency on each core is still satisfied after each rotating step.

We show an example of placing sub-tensors for a 3×3 matrix multiplication (i.e., Equation (1) with $M = K = N = 3$) in Figure 10. Initially, all output sub-tensors (C_j^i) are allocated based on the partitioning plan. Then, the first sub-tensor set $\{A_0^0, A_1^0, A_2^0\}$ is partitioned along the temporal dimension and distributed to the corresponding cores (the first row of cores). Given the current placement of A_i^0 and C_i^0 , we infer that B_i^0 should be placed in the first row of cores due to data dependency. Subsequently, the remaining B_j^i sub-tensors are placed sequentially in each column of cores. Following this process, the placement of all A_j^i sub-tensors can be inferred.

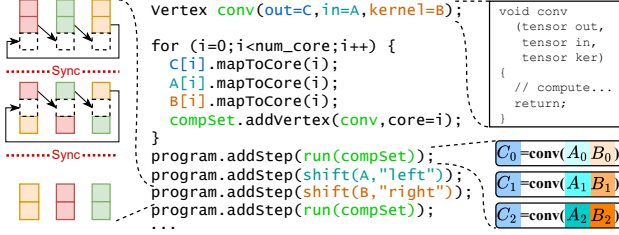


Figure 11. The kernel code example of T10 on IPU.

Sub-operator computation scheduling. After the tensor placement, T10 organizes the computation on each core as nested loops of interleaved compute and shift stages. Each loop corresponds to a temporal partition number. To determine the optimal loop order, T10 designates the dimension belonging to the tensor with the smaller size as the inner loop to reduce the total communication volume, as the inner loop is executed more times. To generate local computations for each core, T10 invokes the corresponding compute function with the partition configuration and the tensor expression.

5 Implementation Details

We implement T10 as a standalone compiler framework and adapt it to Graphcore IPU, a representative accelerator with distributed on-chip memory. T10 takes a DNN model in ONNX format [35] as input and parses it into an operator graph, where each operator is represented by a tensor expression. It performs the intra- and inter-operator optimizations, and outputs the executable kernel code for IPU. The optimization passes are implemented with 4K LoC of Python. The kernel code generation is developed with 4.5K LoC of Python and 1.5K LoC of C++ and IPU assembly [15].

Kernel code generation. Figure 11 shows an example of the code generated for IPU. First, T10 allocates the tensors using the `t.mapToCore(int i)` interface, which maps the sub-tensor `t` to core `i`. And then, T10 uses the `compute` interface to generate the per-step local computation task on each core, which is called `Vertex` on IPU. In each step, each core runs its own `Vertex`. All `Vertices` executed in the same step are homogeneous in T10 and form a `ComputeSet`. T10 schedules data shifts across all cores. After a shift, all cores can execute the same `ComputeSet` with the new local data. Once an operator finishes, the program may rearrange the data on cores if necessary, before launching the next one.

Multi-copy shift with buffer. The IPU does not support the shift interface by default. The key limitation is that the source (data being shifted out) and the destination (data being shifted in) overlap in memory. To overcome this issue, we implement a low-overhead pseudo-shift mechanism as shown in the left side of Figure 11. We use a temporary buffer in each core, as shown by the dashed boxes, to avoid overlapping source and destination. A larger temporary buffer consumes more on-chip memory space, while a smaller buffer will require multiple shift iterations. T10 reserves an 8KB

Table 2. DNN models used in our evaluation.

Name	Description	# of Parameters
BERT [6]	Natural Language Processing	340M
ViT [7]	Transformer-based Vision	86M
ResNet [18]	CNN-based Vision	11M
NeRF [30]	3D Scene Synthesis	24K
OPT [57]	Large Language Model	1.3B to 13B
Llama2 [50]	Large Language Model	7B to 13B
RetNet [46]	State Space Model for Language	1.3B

Table 3. Hardware specifications (per-chip) of the A100 GPU and Graphcore IPU MK2 used in our evaluation.

	A100 GPU [32]	IPU MK2 [24]
Local Cache (total)	20.25MB	896MB
Global Cache	40MB	N/A
Off-chip B/W	2000GB/s	8GB/s
Inter-core B/W	N/A	6GB/s per link
Number of Cores	108	1472
Total FP16 FLOPS	312TFLOPS	250TFLOPS

buffer in each core’s local memory by default, which incurs negligible synchronization overhead, while allowing users to configure the buffer size by themselves.

Compound axis in tensor expressions. T10 supports all common tensor expressions in DNN models, including those with compound axes (i.e., an axis that is a function of other axes). For example, a 2D Convolution can be expressed as

$$O[b, f, h, w] += I[b, c, h + kh, w + kw] * C[f, c, kh, kw], \quad (2)$$

where b is batch size, c/f is the number of input/output channels, h/w is output height/width, and kh/kw is kernel height/width. For the compound axes $h + kh$ and $w + kw$, T10 partitions each basic axis (e.g., h and kh) individually.

Inter-operator transition. T10 allows two consecutive operators to have different tensor partitioning plans. If the input and output tensor layouts do not match, T10 inserts an all-to-all inter-core data exchange operation to adjust the layout. The overhead is typically small compared to the operator execution, since the intermediate tensor size is small compared to the inter-core shift volume during execution.

6 Evaluation

We show that (1) T10 outperforms state-of-the-art DNN compilers on Graphcore IPU by up to $3.3\times$ ($1.69\times$ on average) (§6.2); (2) it enables flexible and efficient intra- and inter-operator scheduling (§6.3 and §6.4); (3) it scales as we increase the number of cores (§6.5); (4) it unleashes the benefit of distributed on-chip memory compared to a popular shared-memory-based AI chip - A100 GPU (§6.6); and (5) it benefits LLMs by alleviating the memory bandwidth wall (§6.7).

6.1 Experimental Setup

We evaluate T10 with DNNs of different types and sizes in Table 2, including CNNs (ResNet), Transformers (BERT and ViT), and fully-connected networks (NeRF). For each model, we test from batch size 1, and double the batch size until the program cannot fit into the on-chip memory. We also evaluate LLM decoding workloads using LLMs like OPT and Llama2 (see §6.7). Our evaluation focuses on inference,

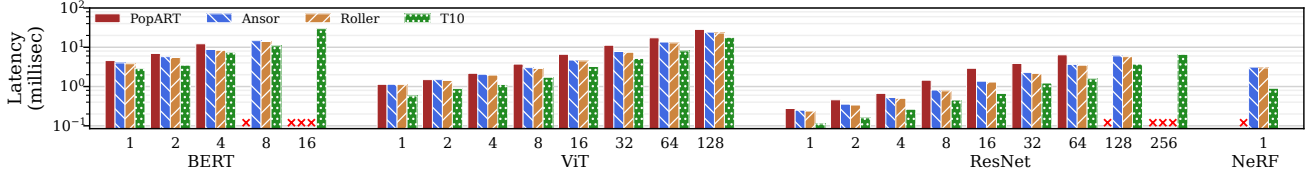


Figure 12. Inference latency of DNN models for various batch sizes. “x” indicates the program cannot fit into an IPU chip.

because the IPU chip we can access has limited on-chip capacity and is mostly used for inference.

We execute models on a Graphcore IPU MK2 chip [24] (see Table 3). We compare T10 with vendor libraries and current DL compilers that support distributed on-chip memory, including Graphcore’s official Poplar Advanced Run Time (PopART) [14] and two DL compilers based on VGM, Ansor [59] (modified to support IPU) and Roller [63]. As T10 only applies lossless optimizations without changing any arithmetic operations in a model, the inference accuracies of T10, Roller, and PopART have negligible differences given the same pretrained parameters.

6.2 End-to-End Performance

As shown in Figure 12, T10 achieves $1.69\times$ end-to-end inference latency improvement on average than Ansor and Roller. T10 supports larger batch sizes and models, while other baselines fail to execute as the batch size gets larger.

DNN inference latency. Ansor and Roller can outperform PopART by $1.33\times$ and $1.38\times$ on average. They improve single-operator performance by using appropriate sub-operator sizes, such that each sub-operator utilizes more local memory, leading to higher data reuse and compute intensity. They have similar performance by exploring the same optimization space. However, they still suffer from significant inter-core communication overhead, due to the VGM abstraction. They also cannot make globally optimized decisions, as they only consider single-operator performance.

T10 outperforms Roller by up to $3.3\times$ ($1.69\times$ on average). It avoids data duplication caused by VGM and efficiently utilizes on-chip memory with intra-operator scheduling, which enables larger sub-operator sizes and reduces the inter-core communication overhead. With the holistic inter-operator scheduling, T10 reduces the setup and execution time of one operator with minimum negative impact on other operators.

T10 also supports larger batch sizes. For example, PopART fails to execute the largest batch size of most models (except ViT) and cannot execute NeRF at all. Even though Roller can run a larger benchmark by selecting smaller sub-operators, it incurs much more performance penalty than T10, as it needs to fetch data frequently from the virtual global memory.

Inference latency breakdown. We break down the compute time and inter-core data transfer time of Roller and T10 in Figure 13. Compared with Roller, T10 reduces the inter-core communication overhead from 50%–74% to only 8%–43%. For most models, while the spatial-reduction nature of common operators like MatMul intrinsically incurs

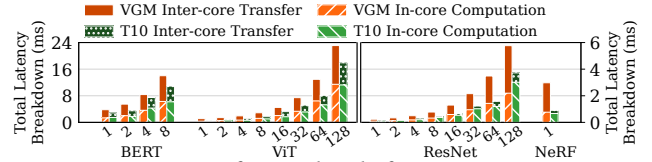


Figure 13. Data transfer overhead of executing various DNN models with different batch sizes on IPU.

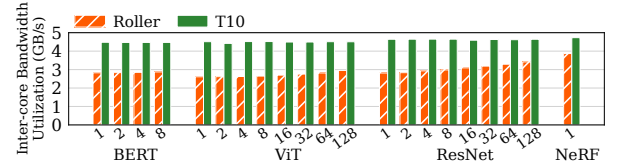


Figure 14. Average inter-core bandwidth utilized by each core when executing various DNN models on IPU.

significant inter-core data sharing, T10 reduces the communication overhead by smartly shifting the shared tensor data. For ResNet and NeRF, T10 automatically minimizes the inter-core movements of their large input activation tensors, by efficiently sharing the smaller convolution kernels or model weights across the cores, enabling even lower communication overhead. Similar patterns and benefits are also observed in the large language models examined in §6.7, where T10 prefers to share the smaller input tensors while keeping the huge model weights stationary on each core.

Inter-core bandwidth utilization. To further explain T10’s low communication overhead, we show the average inter-core bandwidth utilized by each core during inter-core data transfers in Figure 14. While T10 uses an average bandwidth of 4.42GB/s to 4.73GB/s per core when running different DNN models (5.5GB/s is the advertised roofline), Roller only utilizes 2.61GB/s to 3.87GB/s, due to the inter-core communication inefficiencies discussed in §2.2. Notably, models that shift more data each step (e.g., NeRF) have higher utilization.

T10 operator performance. T10 improves the single operator performance by up to $10.79\times$ (ResNet-BS8) compared to Roller. As T10 reduces the setup overhead of performance-critical operators, its inter-operator memory reconciliation introduces minimal negative impact on other operators (§4.3.2). T10 improves the performance of more than 80% of the operators at the cost of slowing down less than 10% (Figure 15).

T10 compilation time. Figure 16 shows the compilation time of T10 for different models and batch sizes. T10 finishes compilation in a few hours for most DNN inference programs (tested with AMD Ryzen 7950X3D). As T10 exploits the predictability of the hardware architecture with the cost model

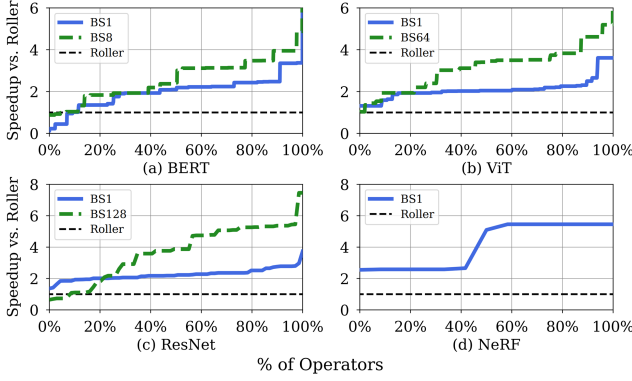


Figure 15. Distribution of T10’s operator performance vs. Roller. We plot the min. and max. batch sizes as examples.

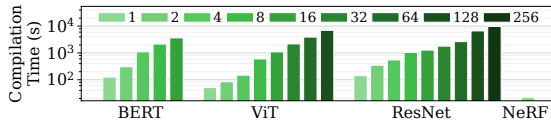


Figure 16. T10 compilation time of different batch sizes.

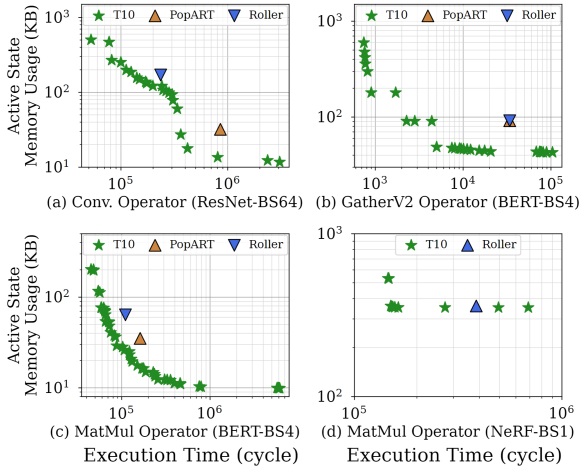


Figure 17. Candidate execution plans of representative operators (e.g., Figure (a) is a convolution operator from ResNet with batch size 32). Stars are optimal execution plans found by T10. Triangles are the plans used by PopART and Roller, respectively. PopART fails to execute NeRF in Figure (d).

and search constraints (§4.3.1 and §5), it avoids the expensive profiling for tuning each operator (e.g., Ansor [59]).

6.3 Analysis of Intra-operator Optimization

We now examine T10’s intra-operator optimizations (§4.3.1). **Flexibility of intra-operator plan selection.** T10’s intra-operator scheduling facilitates flexible inter-operator scheduling. Figure 17 shows the set of optimal execution plans found by T10 for the representative operators compared to the plans used by PopART and Roller. The selected operators represent the majority of computation in a DNN workload.

For most operators, T10’s search space always contains an execution plan that is both faster and more memory efficient

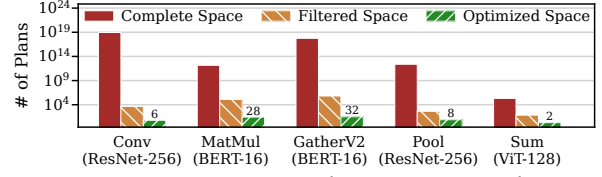


Figure 18. Intra-operator search space sizes. Each operator “Op (Model-BS)” is selected from Model with batch size BS.

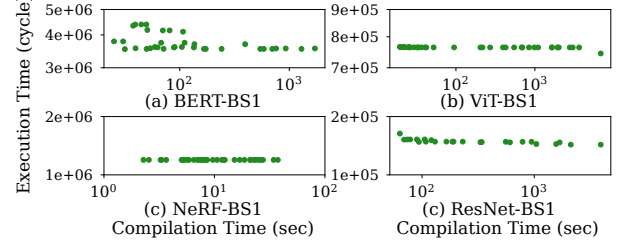


Figure 19. T10 compilation time and resulting execution latency with different constraint settings.

than PopART. Roller always tries to find the fastest plan that utilizes the most per-core local memory. However, Roller’s maximum memory usage is limited due to the virtual global memory region (Figure 2). By removing the global memory, T10 enables a larger active memory and allows faster plans. Roller cannot make a globally optimized plan that exploits the trade-off between operators. In contrast, T10 maintains a set of Pareto-optimal plans and enables the inter-operator scheduling for optimized end-to-end performance.

Intra-operator search space size reduction. To show how T10 explores the large search space of a single operator, we break down the search process and compare the remaining search space size after each critical step. Figure 18 compares (1) the *Complete Space* of all execution plans; (2) the *Filtered Space* after applying the constraints defined in §5 but before applying the cost model; and (3) the *Optimized Space* containing the Pareto-optimal plans selected by the cost model. In Figure 18, Conv, MatMul, GatherV2 are the operators whose intra-operator optimizations generate the largest search space and contribute the most compile time, compared to other operators from the evaluated models in Table 2.

The complete space grows exponentially with the number of dimensions in an operator (up to 10^{19} plans for the largest convolution in ResNet, which has 7 dimensions). The search constraints narrow down the search space to less than 10^4 plans. As the efficient cost model in §4.3.1 takes less than 100 milliseconds to evaluate each plan on one CPU core, T10 can explore 10^4 plans in 30 seconds using 32 CPU cores. The final number of the Pareto-optimal plans is less than 50 for most operators, and each operator’s final plans can be cached and reused for identical operators within or across model(s).

Compilation time with different constraint settings. To compile faster, users can set stricter intra-operator search constraints (see §5), which decreases the filtered space size and compilation time. Figure 19 shows the trade-off between

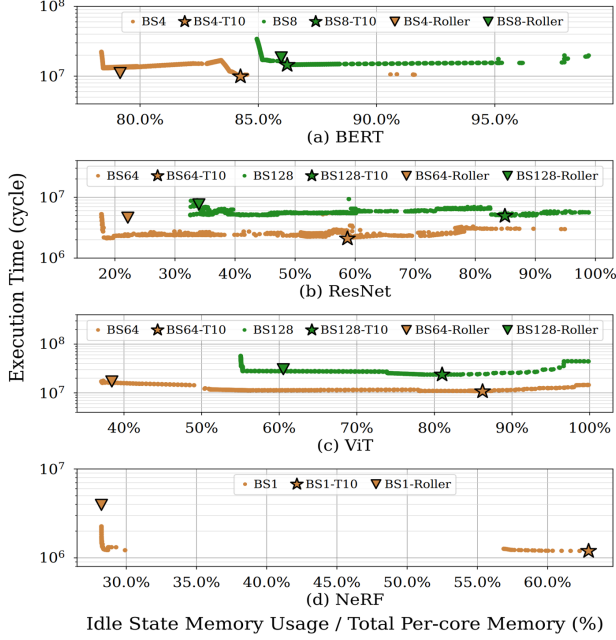


Figure 20. End-to-end execution plans. BS# is batch size #. Dots are plans explored by T10. Stars and triangles are the final plans selected by T10 and Roller, respectively.

execution performance and compilation time. For most models, a strict constraint setting that takes only one minute to compile, already yields near-optimal performance.

6.4 Analysis of Inter-operator Optimization

We visualize the inter-operator search process by plotting the end-to-end execution plan explored at each search step in Figure 20. For most benchmarks, Roller generates the slowest (e.g., left-most) plan that requires the least idle state memory, which incurs significant operator setup time. This is because Roller does not optimize the idle-to-active state setup overhead of operators. With inter-operator memory reconciliation (§4.3.2), T10 finds a globally optimized execution plan by recognizing the *minimum active state memory demand*. For example, for ResNet-BS64 in Figure 20 (b), T10 expands the idle state memory to about 58% of the total on-chip memory by performing the setup phase for the performance-critical operators in advance. This improves end-to-end performance because many operators only demand at most 40% of total memory as the active state memory.

6.5 Scalability of T10

As the process node technology advances, we expect an intelligence processor will scale with more cores on a chip. We emulate a larger chip by deploying T10 on a Virtual IPU (V-IPU) [16], which exposes the 2 or 4 interconnected IPU chips on the same board to the compiler as a single IPU chip with 2,944 or 5,888 cores. The inter-chip communication bandwidth is 160GB/s. We also emulate smaller chips by restricting the number of cores in our compiler.

Figure 21 compares the execution performance of Roller and T10 on IPU devices with different numbers of cores. T10 always outperforms Roller. While both compilers achieve faster computation with more cores, T10 scales much better than Roller in terms of the inter-core data transfer, for two reasons. First, the *rTensor* abstraction eliminates the imbalanced and redundant inter-core data accesses caused by the VGM. Second, T10’s inter-core scheduling policy utilizes the on-chip memory more efficiently to decrease the data transfer volume and the setup time of large operators. In most cases, T10 achieves similar or even better performance than Roller while using fewer cores. For example, T10 enables NeRF and ResNet with small batch sizes (BS-1 and BS-2) to be 2× faster than Roller while using only half of the cores.

With more than one chip, the inter-core communications in V-IPUs are bottlenecked by the inter-chip IPU-Link, causing the average effective inter-core bandwidth to drop by 26%-33%. In this case, the execution time with Roller may even increase (e.g., ResNet BS-1) when we use more than one chip. In contrast, T10 does not increase the data transfer overhead despite the inter-chip communication overhead.

6.6 Comparison with A100 GPU

To show how T10 unleashes the potential of inter-core connected architecture, we compare an IPU MK2 chip against an A100 GPU, as both chips use 7nm technology and have similar FLOPS (Table 3). We run models with TensorRT [34] in an Azure NC24ads_A100_v4 instance. We use FP16 and TensorCores on A100. We warm up the models such that all data are in the HBM before running the experiments.

While Figure 22 shows that TensorRT on A100 outperforms PopART, Ansor, and Roller on IPU in Figure 12, T10 allows IPU to outperform A100 with small batch sizes by up to 2.44×. T10 is especially good at small batches since it can efficiently utilize the fast on-chip memory of IPU, while the A100 is bottlenecked by loading data from off-chip memory.

As batch size increases, the compute intensity increases, so the execution time becomes bounded by the peak FLOPS. Meanwhile, larger batch sizes also increase memory usage, leaving less space in the on-chip memory for T10 to trade off for less communication overhead. Due to the lower peak TFLOPS of IPU and the increased communication overhead when on-chip memory is nearly full, IPU suffers inferior performance in certain scenarios. As intelligence processors continue to evolve, we believe they are a promising option for future latency-sensitive DNN workloads with T10 support.

6.7 Performance of T10 for LLMs

As Large Language Models (LLMs) become popular, inter-core connected DNN processors spot a new area to release their potential on high aggregated inter-core bandwidth. We examine two LLMs with standard transformer (OPT [57] and Llama2 [50]) and one LLM (RetNet [46]) with new transformer architecture designed for better scalability and memory-efficiency [17, 38, 46]. As the IPU chip we can access has

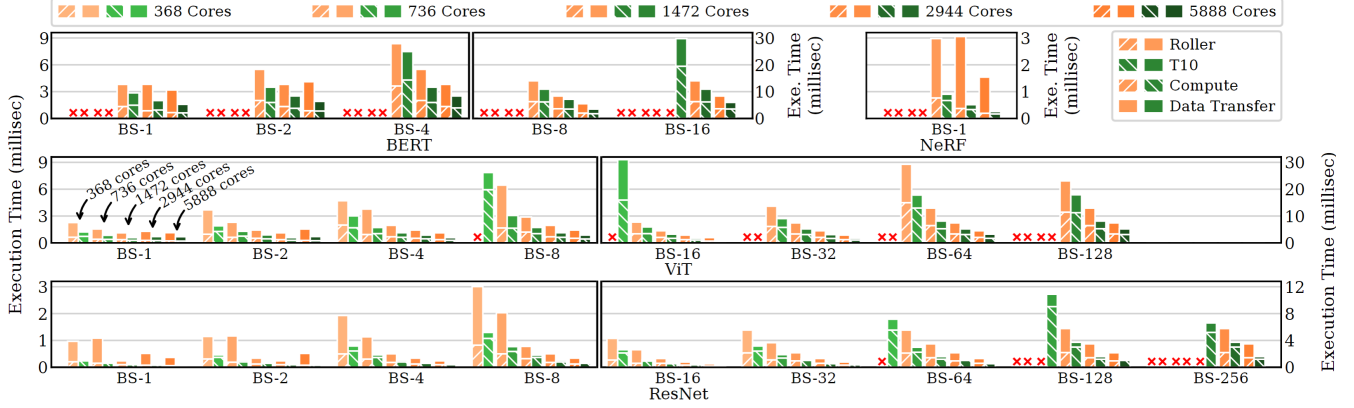


Figure 21. Performance of IPU with different number of cores. The $\times$ indicates the DNN model cannot fit into the IPU chip.

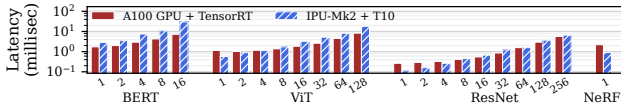


Figure 22. Inference latency of IPU+T10 vs. A100+TensorRT.

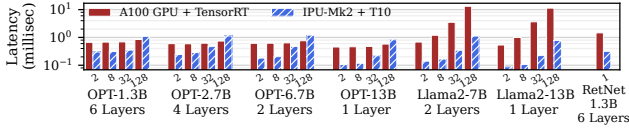


Figure 23. Inference latency of IPU+T10 vs. A100+TensorRT on large language models with different batch sizes.

insufficient on-chip memory to fit an entire LLM, we run a subset of layers for each LLM. To run an entire LLM, users may follow common approaches that connect multiple chips as a pipeline [11, 37], where T10 reduces the latency and number of required chips by optimizing the execution and memory usage on each chip. The entire LLM’s performance can be inferred from the single-chip performance: since the intermediate data between layers is small (e.g., 131KB per token for Llama2-13B), the inter-chip communication overhead between pipeline stages is negligible.

We compare the execution of LLM layers with A100 GPU in Figure 23. Thanks to T10 and the fast inter-core connection, IPU achieves up to 16.38 $\times$ lower inference latency (3.10 $\times$ on average) than A100. By replacing virtual global memory with r Tensor abstraction (§2.2), T10 frees up sufficient memory space for the large operators in LLMs. A100 is limited by its 40MB global cache, which cannot fit a single large operator on chip. This requires A100 to load each model parameter from the slow off-chip memory multiple times. Thus, the HBM bandwidth significantly bottlenecks the GPU performance for small batch sizes. For large batch sizes, similar to the case in §6.6, both GPU and IPU become compute-bound, and IPU suffers from its lower peak FLOPS.

6.8 Performance of T10 with Off-chip HBM

Given the absence of HBM on IPU, it is inefficient to serve LLMs on a single IPU chip, since it takes significant amount of time to load model parameters with off-chip memory (8GB/s). But if we combine the distributed on-chip memory with a large HBM, we can take advantage of both inter-core connection’s high bandwidth and the HBM’s large capacity.

We emulate HBM with different bandwidths on IPU by delaying the operator execution according to the predicted time of loading the operator from HBM using the roofline model [53]. We extend T10 to support HBM. We enable double buffering to overlap operator execution and HBM data transfer. The buffer size for execution/prefetching is 596MB/298MB, determined empirically based on the operator sizes. We examine two cases: (1) *Single Op*: we execute an operator and prefetch the next operator simultaneously; (2) *Inter Op*: we prefetch multiple operators as a group while the current group is executing. We apply T10’s inter-op scheduling to decide the idle memory size for each operator. We ensure the minimum total memory requirement of the operator group is less than the prefetch buffer size.

For the *Single Op* case, Roller and T10 have similar performance when the HBM bandwidth is limited (Figure 24), as the execution is bottlenecked by the HBM. When the HBM bandwidth increases, the execution becomes compute-bound, T10 performs better, thanks to the execution plan identified by its intra-operator search. For the *Inter Op* case, both T10 and Roller perform better when the HBM bandwidth is low, because grouping operators with different compute intensities helps balance the execution and prefetching. With inter-operator scheduling, T10 achieves more benefits from grouping operators than Roller. As we further increase the HBM bandwidth, the execution is compute-bound, and *Inter Op* is slightly slower than *Single Op*, since operators in the same group compete for the on-chip memory capacity.

7 Discussion and Future Work

Apply T10 to multiple chips. While T10 focuses on a single chip, it can be extended to optimize the inter-chip

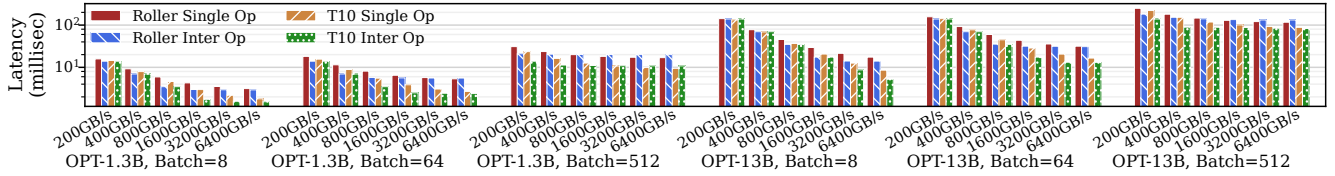


Figure 24. Emulated execution time of Roller and T10 with different HBM bandwidths.

communication in modern AI infrastructures. T10 can trade-off between the inter-chip communication overhead and the per-chip memory consumption, enabling hardware clusters to execute larger DL workloads with higher performance.

Other inter-core connected hardware. The inter-core connection represents an important trend in AI chip evolution. As large DL models are increasingly bounded by memory, new GPU architectures like Hopper [33] also introduce inter-core links to connect the stream multiprocessors (SMs) into “thread block clusters”. More inter-SM data sharing allows less off-chip data access, which enables faster execution and consumes less energy. T10 can be extended to optimize the inter-core communication in these new architectures.

Combine IPU with HBM. Since the IPU device we can access is not equipped with HBM, we emulate HBM on IPU in §6.8. However, an inter-core connected chip recently released by SambaNova has HBM installed [40]. Similarly, IPU can support HBM by attaching HBM controllers to its on-chip interconnect, where HBM controllers can deliver data to cores in the same way as inter-core transfers.

8 Related Work

Deep learning compiler. DL compilers often focus on three design aspects: (1) intermediate representation (IR), which represents a computation workload; (2) computation model, which maps the computation to hardware; and (3) optimization, which identifies and explores the optimization space.

T10 uses the same IR with existing DL compilers: it uses operator graph [35] to represent a DNN, and uses tensor expression [51] to represent each operator. As existing DL compilers [2, 5, 26, 36, 48, 59, 63] are designed for the shared memory architecture, they use the “load-compute-store” computation model. To explore this model’s optimization space, they deploy techniques like tiling [5, 26, 48, 59, 63], loop reordering [5, 26], polyhedral model [5, 48], load-compute overlapping [36], and sparsity [2]. As T10 targets a new distributed memory architecture, it uses a new “compute-shift” model and explores the new optimization space with new techniques. T10 may work in orthogonal with other existing optimization techniques originally developed on the shared memory architecture, such as kernel fusion [48, 58, 62], parallel kernel packing [29], and iteration batching [8, 56].

Distributed model partitioning. Many DL frameworks partition computation over distributed nodes with model parallelism, such as JAX [3], PartIR [1], GSPMD [54], Megatron

[31], Alpa [60], FlexFlow [20], and Tofu [52]. They adopt a single-program-multiple-data (SPMD) framework to shard an operator into multiple parallel sub-tasks, and insert a communication stage (e.g. AllReduce) when necessary to merge the sharded result. The two stages are often optimized separately. Also, to ensure each sub-task has complete local input data, some tensors will be replicated to multiple nodes, causing increased memory consumption. In contrast, T10 schedules an operator as multiple interleaved compute and communication stages, and optimizes the stages holistically.

Dataflow architectures. Prior works facilitate the execution of various workloads on dataflow architectures. DISTAL [55] and Tenet [28] provide scheduling primitives for users to write customized dataflow plans for linear algebra operations. SambaNova [41] maps DNN execution to a dataflow accelerator using a mix of model and pipeline parallelisms, which may increase both latency (due to pipeline) and memory consumption (due to data replication). Existing dataflow compilers [9, 10, 25, 43, 47] also focus on mapping general-purpose computation to interconnected CPU cores. In contrast, T10 targets AI chips with thousands of fully connected tensor cores, and it can automatically compile an end-to-end DNN model into a compute-shift program with optimized execution latency. The compute-shift model allows more generality and flexibility for diverse tensor operations than conventional hardware-defined systolic arrays [23, 61].

9 Conclusion

We present T10, an end-to-end deep learning compiler for inter-core connected intelligence processors. It generalizes the compute-shift computing paradigm on distributed on-chip memory for enabling efficient operator partitioning and cost-aware operator scheduling. We show its efficiency and scalability on a real massively-parallel intelligence processor.

Acknowledgments

We thank the anonymous reviewers and our shepherd Byung-Gon Chun for their insightful comments and feedback. We thank the members in the Systems Platform Research Group at University of Illinois Urbana-Champaign for discussing and proofreading our paper. They include Benjamin Reidys, Haoyang Zhang, Shaobo Li, Daixuan Li, Eric Zhou, and Noelle Crawford. We thank Chen Jin, Chengshun Xia, and Han Zhao for the technical support on Graphcore IPU. This work was partially supported by NSF CAREER CNS-2144796.

References

- [1] Sami Alabed, Daniel Belov, Bart Chrzaszcz, Juliana Franco, Dominik Grewe, Dougal Maclaurin, James Molloy, Tom Natan, Tamara Norman, Xiaoyue Pan, Adam Paszke, Norman A. Rink, Michael Schaarschmidt, Timur Sitdikov, Agnieszka Swietlik, Dimitrios Vytiniotis, and Joel Wee. 2024. PartIR: Composing SPMD Partitioning Strategies for Machine Learning. *arXiv preprint arXiv:2401.11202* (2024).
- [2] Manya Bansal, Olivia Hsu, Kunle Olukotun, and Fredrik Kjolstad. 2023. Mosaic: An Interoperable Compiler for Tensor Algebra. *Proc. ACM Program. Lang.* (2023).
- [3] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. 2018. *JAX: composable transformations of Python+NumPy programs*. <http://github.com/google/jax>.
- [4] Cerebras. 2021. Cerebras Systems Raises \$250M in Funding for Over \$4B Valuation to Advance the Future of Artificial Intelligence Compute. <https://www.cerebras.net/press-release/cerebras-systems-raises-250m-in-funding-for-over-4b-valuation-to-advance-the-future-of-artificial-intelligence-compute/>.
- [5] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805* (2019).
- [7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *arXiv preprint arXiv:2010.11929* (2021).
- [8] FriendliAI. 2024. FriendliAI. <https://friendli.ai/>.
- [9] Michael I. Gordon, William Thies, and Saman Amarasinghe. 2006. Exploiting coarse-grained task, data, and pipeline parallelism in stream programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XII)*.
- [10] Michael I. Gordon, William Thies, Michal Karczmarek, Jasper Lin, Ali S. Meli, Andrew A. Lamb, Chris Leger, Jeremy Wong, Henry Hoffmann, David Maze, and Saman Amarasinghe. 2002. A stream compiler for communication-exposed architectures. In *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS X)*.
- [11] Graphcore. 2022. Bow Pod256. <https://www.graphcore.ai/products/bow-pod256>.
- [12] Graphcore. 2022. Graphcore raises \$222 million in series E funding round. <https://www.graphcore.ai/posts/graphcore-raises-222-million-in-series-e-funding-round>.
- [13] Graphcore. 2022. IPU Hardware Overview. [https://docs.graphcore.ai/projects/ipu-programmers-](https://docs.graphcore.ai/projects/ipu-programmers-guide/en/latest/about_ipu.html)
[guide/en/latest/about_ipu.html](https://docs.graphcore.ai/projects/ipu-programmers-guide/en/latest/about_ipu.html).
- [14] Graphcore. 2022. PopART User Guide. <https://docs.graphcore.ai/projects/popart-user-guide/en/latest/intro.html>.
- [15] Graphcore. 2022. Tile Vertex ISA. https://docs.graphcore.ai/projects/isa/en/latest/_static/Tile-Vertex-ISA_1.2.3.pdf.
- [16] Graphcore. 2023. V-IPU User Guide. <https://docs.graphcore.ai/projects/vipu-user/en/latest/introduction.html>.
- [17] Albert Gu and Tri Dao. 2023. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv preprint arXiv:2312.00752* (2023).
- [18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. *arXiv preprint arXiv:1512.03385* (2015).
- [19] Zhe Jia, Blake Tillman, Marco Maggioni, and Daniele Paolo Scarpazza. 2019. Dissecting the Graphcore IPU Architecture via Microbenchmarking. *arXiv preprint arXiv:1912.03413* (2019).
- [20] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2018. Beyond Data and Model Parallelism for Deep Neural Networks. *arXiv preprint arXiv:1807.05358* (2018).
- [21] Norman P. Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, Sheng Li, Peter Ma, Xiaoyu Ma, Thomas Norrie, Nishant Patil, Sushma Prasad, Cliff Young, Zongwei Zhou, and David Patterson. 2021. Ten lessons from three generations shaped Google's TPUV4i. In *Proceedings of the 48th Annual International Symposium on Computer Architecture (ISCA '21)*.
- [22] Norman P. Jouppi, Doe Hyun Yoon, George Kurian, Sheng Li, Nishant Patil, James Laudon, Cliff Young, and David Patterson. 2020. A Domain-Specific Supercomputer for Training Deep Neural Networks. *Commun. ACM* (2020).
- [23] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA '17)*.
- [24] Simon Knowles. 2021. Graphcore Colossus Mk2 IPU. In *2021 IEEE Hot Chips 33 Symposium (HCS '21)*.
- [25] David Koeplinger, Matthew Feldman, Raghu Prabhakar, Yaqi Zhang, Stefan Hadjis, Ruben Fiszal, Tian Zhao, Luigi Nardi, Ardavan Pedram, Christos Kozyrakis, and Kunle Olukotun.

2018. Spatial: a language and compiler for application accelerators. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '18)*.
- [26] Hyoukjun Kwon, Prasanth Chatarasi, Vivek Sarkar, Tushar Krishna, Michael Pellauer, and Angshuman Parashar. 2020. MAESTRO: A Data-Centric Approach to Understand Reuse, Performance, and Hardware Cost of DNN Mappings. *IEEE Micro* (2020).
- [27] Sean Lie. 2021. Multi-Million Core, Multi-Wafer AI Cluster. In *2021 IEEE Hot Chips 33 Symposium (HCS '21)*.
- [28] Liqiang Lu, Naiqing Guan, Yuyue Wang, Liancheng Jia, Zizhang Luo, Jieming Yin, Jason Cong, and Yun Liang. 2021. TENET: A Framework for Modeling Tensor Dataflow Based on Relation-Centric Notation. In *Proceedings of the 48th Annual International Symposium on Computer Architecture (ISCA '21)*.
- [29] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. 2020. Rammer: Enabling Holistic Deep Learning Compiler Optimizations with rTasks. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*.
- [30] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. 2021. NeRF: representing scenes as neural radiance fields for view synthesis. *Commun. ACM* (2021).
- [31] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient large-scale language model training on GPU clusters using megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*.
- [32] Nvidia. 2020. NVIDIA A100 Tensor Core GPU Architecture. <https://resources.nvidia.com/en-us/genomics-ep/ampere-architecture-white-paper>.
- [33] NVIDIA. 2022. NVIDIA Hopper Architecture In-Depth. <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>.
- [34] NVIDIA. 2023. NVIDIA TensorRT. <https://developer.nvidia.com/tensorrt>.
- [35] ONNX. 2023. ONNX. <https://onnx.ai/>.
- [36] OpenAI. 2021. Introducing Triton: Open-source GPU programming for neural networks. <https://openai.com/index/triton/>.
- [37] Dylan Patel and Daniel Nishball. 2023. Groq Inference Tokenomics: Speed, But At What Cost. <https://www.semianalysis.com/p/groq-inference-tokenomics-speed-but>.
- [38] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartłomiej Koptyra, Hayden Lau, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Xiangru Tang, Bolun Wang, Johan S. Wind, Stanislaw Wozniak, Ruichong Zhang, Zhenyuan Zhang, Qihang Zhao, Peng Zhou, Jian Zhu, and Rui-Jie Zhu. 2023. RWKV: Reinventing RNNs for the Transformer Era. *arXiv preprint arXiv:2305.13048* (2023).
- [39] Raghu Prabhakar and Sumti Jairath. 2021. SambaNova SN10 RDU: Accelerating Software 2.0 with Dataflow. In *2021 IEEE Hot Chips 33 Symposium (HCS '21)*.
- [40] Raghu Prabhakar, Ram Sivaramakrishnan, Darshan Gandhi, Yun Du, Mingran Wang, Xiangyu Song, Kejie Zhang, Tianren Gao, Angela Wang, Karen Li, Yongning Sheng, Joshua Brot, Denis Sokolov, Apurv Vivek, Calvin Leung, Arjun Sabnis, Jiayu Bai, Tuowen Zhao, Mark Gottscho, David Jackson, Mark Luttrell, Manish K. Shah, Edison Chen, Kaizhao Liang, Swayambhoo Jain, Urmish Thakker, Dawei Huang, Sumti Jairath, Kevin J. Brown, and Kunle Olukotun. 2024. SambaNova SN40L: Scaling the AI Memory Wall with Dataflow and Composition of Experts. *arXiv preprint arXiv:2405.07518* (2024).
- [41] Raghu Prabhakar, Yaqi Zhang, David Koeplinger, Matt Feldman, Tian Zhao, Stefan Hadjis, Ardavan Pedram, Christos Kozyrakis, and Kunle Olukotun. 2017. Plasticine: A reconfigurable architecture for parallel patterns. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA '17)*.
- [42] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '13)*.
- [43] Alexander Rucker, Shiv Sundram, Coleman Smith, Matthew Vilim, Raghu Prabhakar, Fredrik Kjolstad, and Kunle Olukotun. 2024. Revet: A Language and Compiler for Dataflow Threads. *arXiv preprint arXiv:2302.06124* (2024).
- [44] SambaNova. 2021. SambaNova Systems Becomes World's Best-Funded AI Startup. <https://sambanova.ai/press/series-d>.
- [45] Yining Shi, Zhi Yang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Ziming Miao, Yuxiao Guo, Fan Yang, and Lidong Zhou. 2023. Welder: Scheduling Deep Learning Memory Access via Tilegraph. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI '23)*.
- [46] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. 2023. Retentive Network: A Successor to Transformer for Large Language Models. *arXiv preprint arXiv:2307.08621* (2023).
- [47] M.B. Taylor, J. Psota, A. Saraf, N. Shnidman, V. Strumpen, M. Frank, S. Amarasinghe, A. Agarwal, W. Lee, J. Miller, D. Wentzlaff, I. Bratt, B. Greenwald, H. Hoffmann, P. Johnson, and J. Kim. 2004. Evaluation of the Raw microprocessor: an exposed-wire-delay architecture for ILP and streams. In *Proceedings of the 31st Annual International Symposium on Computer Architecture (ISCA '04)*.
- [48] TensorFlow. 2023. XLA. <https://www.tensorflow.org/xla>.
- [49] Tenstorrent. 2023. Meet Grayskull. <https://tenstorrent.com/grayskull/>.
- [50] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne

- Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288* (2023).
- [51] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S. Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2018. Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions. *arXiv preprint arXiv:1802.04730* (2018).
- [52] Minjie Wang, Chien-chin Huang, and Jinyang Li. 2019. Supporting Very Large Models Using Automatic Dataflow Graph Partitioning. In *Proceedings of the Fourteenth EuroSys Conference 2019 (EuroSys '19)*.
- [53] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* (2009).
- [54] Yuanzhong Xu, HyoukJoong Lee, Dehao Chen, Blake Hechtman, Yanping Huang, Rahul Joshi, Maxim Krikun, Dmitry Lepikhin, Andy Ly, Marcello Maggioni, Ruoming Pang, Noam Shazeer, Shibo Wang, Tao Wang, Yonghui Wu, and Zhifeng Chen. 2021. GSPMD: General and Scalable Parallelization for ML Computation Graphs. *arXiv preprint arXiv:2105.04663* (2021).
- [55] Rohan Yadav, Alex Aiken, and Fredrik Kjolstad. 2022. DISTAL: the distributed tensor algebra compiler. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '22)*.
- [56] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*.
- [57] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. *arXiv preprint arXiv:2205.01068* (2022).
- [58] Jie Zhao, Siyuan Feng, Xiaoqiang Dan, Fei Liu, Chengke Wang, Sheng Yuan, Wenyuan Lv, and Qikai Xie. 2023. Effectively Scheduling Computational Graphs of Deep Neural Networks toward Their Domain-Specific Accelerators. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI '23)*.
- [59] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: generating high-performance tensor programs for deep learning. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation (OSDI '20)*.
- [60] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*.
- [61] Size Zheng, Renze Chen, Anjiang Wei, Yicheng Jin, Qin Han, Liqiang Lu, Bingyang Wu, Xiuhong Li, Shengen Yan, and Yun Liang. 2022. AMOS: Enabling automatic mapping for Tensor Computations on spatial Accelerators with Hardware Abstraction. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA '22)*.
- [62] Zhen Zheng, Xuanda Yang, Pengzhan Zhao, Guoping Long, Kai Zhu, Feiwen Zhu, Wenyi Zhao, Xiaoyong Liu, Jun Yang, Jidong Zhai, Shuaiwen Leon Song, and Wei Lin. 2022. AStitch: Enabling a New Multi-Dimensional Optimization Space for Memory-Intensive ML Training and Inference on Modern SIMT Architectures. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*.
- [63] Hongyu Zhu, Ruofan Wu, Yijia Diao, Shanbin Ke, Haoyu Li, Chen Zhang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Wei Cui, Fan Yang, Mao Yang, Lidong Zhou, Asaf Cidon, and Gennady Pekhimenko. 2022. ROLLER: Fast and Efficient Tensor Compilation for Deep Learning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*.